

FPT UNIVERSITY — LAB211

ĐỀ TÀI THỰC HÀNH



Stadium Ticket Booking Simulation

Hệ thống Mô phỏng Bán Vé Sân Vận Động

BIG QUESTION:

"Cơ chế đồng bộ hóa nào đảm bảo không xảy ra Double Booking khi hàng nghìn Fan Threads cùng đặt vé cùng một lúc?"

Mã môn học LAB211 — OOP with Java	Kiến trúc bắt buộc MVC (Model-View-Controller)
Cấp độ Java OOP Nâng cao — Semester 3	Dữ liệu File CSV — tổng ≥ 10.000 dòng
Thời gian thực hiện 10 tuần (project nhóm)	Sản phẩm nộp Code + Doc + Slide + AI Log

⚠ LƯU Ý QUAN TRỌNG

Đây là project nhóm (2-4 thành viên). Mỗi thành viên phải có AI Log riêng ghi lại toàn bộ tương tác với AI trong quá trình làm dự án — đây là cơ sở cho phần AI Reflection (30% tổng điểm).

1. CÂU HỎI NGHIÊN CỨU (BIG QUESTION)



RESEARCH QUESTION

"Trong kịch bản hàng nghìn người hâm mộ (Fan Threads) cùng lúc đặt mua vé cho cùng một khu vực khán đài trên file CSV — cơ chế đồng bộ hóa nào đảm bảo không xảy ra tình trạng Double Booking và tối ưu hóa tốc độ giao dịch để fan không phải chờ đợi quá lâu?"

1.1. Tại sao đây là vấn đề quan trọng?

Trong hệ thống bán vé thực tế (TicketMaster, Shopee Tickets...), vấn đề Double Booking là rủi ro nghiêm trọng nhất:

- **Double Booking:** Ghế A-Row3-Seat12 được bán cho Fan #001 và Fan #002 cùng lúc — cả hai đều có vé hợp lệ nhưng chỉ một người được vào sân.
- **Root cause:** Thao tác "đọc trạng thái ghế → kiểm tra → ghi vé" không phải atomic trên file CSV khi nhiều thread cùng truy cập.
- **Trade-off:** Khóa quá chặt → throughput thấp, fan chờ lâu. Khóa quá lỏng → double booking xảy ra.

1.2. Research Question Lite — Dành cho phần Báo cáo



RESEARCH QUESTION (LITE) — so sánh thực nghiệm

"Transaction isolation level nào ngăn được race condition âm kho trong Flash Sale — và đánh đổi gì về throughput?"

Nhóm phải implement và so sánh ít nhất 3 cơ chế sau bằng Simulator Tool:

- ...

Kết quả: biểu đồ so sánh throughput (vé/giây) vs. double booking rate (%) theo từng cơ chế.

2. TỔNG QUAN BÀI TOÁN

Nhóm xây dựng hệ thống phần mềm mô phỏng quy trình bán vé trực tuyến cho các trận bóng đá tại sân vận động. Toàn bộ dữ liệu (sân vận động, ghế ngồi, fan, vé, giao dịch) được lưu trữ dưới dạng file CSV. Hệ thống phải đảm bảo tính toàn vẹn dữ liệu — đặc biệt là ngăn chặn Double Booking — trong điều kiện nhiều người dùng đặt vé đồng thời.

2.1. Bối cảnh nghiệp vụ

- Sân vận động gồm nhiều khu vực (Section): VIP, Thường, Hạng phổ thông, Đứng
- Mỗi khu vực chia thành nhiều Hàng (Row), mỗi hàng có nhiều Ghế (Seat)
- Mỗi ghế có 3 trạng thái: AVAILABLE → LOCKED (đang chọn) → BOOKED
- Một ghế BOOKED không thể được bán lại — đây là ràng buộc toàn vẹn cốt lõi
- Fan đăng ký tài khoản và có thể đặt tối đa 4 vé/lần giao dịch
- Dữ liệu toàn bộ lưu trong file CSV — không dùng database

2.2. Các đối tượng dữ liệu chính

Entity (Model)	Thuộc tính chính	File CSV tương ứng
Stadium		stadiums.csv
Section		sections.csv
Seat		seats.csv (~10k dòng)
Match		matches.csv
Fan		fans.csv
Ticket		tickets.csv
BookingTransaction		transactions.csv

 **Lưu ý về trường `version` trong Seat:** Đây là số nguyên tăng dần mỗi lần ghế được cập nhật trạng thái. Dùng cho cơ chế Optimistic Locking — thread chỉ được ghi nếu version đọc ra = version hiện tại trong file.

3. KIẾN TRÚC MVC BẮT BUỘC

⚠ **BẮT BUỘC:** Toàn bộ hệ thống phải tuân thủ mô hình MVC. Không được phép trộn logic nghiệp vụ vào View, không được phép truy cập file trực tiếp từ Controller mà không qua Model layer.

3.1. Tổng quan phân lớp MVC

Lớp	Trách nhiệm	Được phép làm	KHÔNG được làm
Model	Dữ liệu + nghiệp vụ cốt lõi	Đọc/ghi CSV, validate business rules, quản lý trạng thái seat	Hiển thị UI, nhận input từ bàn phím
View	Hiển thị + nhận input	In menu, bảng, nhận input từ người dùng, gọi Controller	Đọc/ghi file, tính toán nghiệp vụ
Controller	Điều phối flow	Nhận request từ View, gọi Model, trả kết quả về View	Trực tiếp đọc/ghi file, hiển thị UI

3.2. Lược đồ Class — Toàn bộ hệ thống

Sơ đồ dưới mô tả các class và mối quan hệ. Nhóm phải vẽ lại thành UML Class Diagram đầy đủ (có attributes, methods, visibility, multiplicity) trong phần báo cáo.

MODEL Layer

REPOSITORY — Tầng truy cập File CSV

CONTROLLER Layer

VIEW Layer

4. FLOWCHART CÁC LUỒNG CHÍNH

Nhóm phải vẽ các flowchart dưới đây bằng tool (draw.io, Lucidchart, PlantUML) và nộp kèm báo cáo. Mô tả bên dưới là ngôn ngữ tự nhiên — flowchart thực tế cần có hình thời quyết định, hình chữ nhật xử lý, và mũi tên điều kiện.

4.1. Luồng Đặt Vé (Booking Flow)

4.2. Luồng Ngăn chặn Double Booking (Synchronization Flow)

4.3. Luồng Simulator Tool

4.4. Luồng Generate CSV Data (10.000 dòng)

5. HƯỚNG DẪN IMPLEMENTATION

6. LỘ TRÌNH THỰC HIỆN — 10 TUẦN

Tuần	Milestone	Công việc chi tiết	Deliverable
T1–T2	 Phân tích & Dữ liệu	Phân tích yêu cầu, vẽ Use Case Diagram. Thiết kế class diagram chi tiết. Thiết kế schema CSV (cột, kiểu dữ liệu, quan hệ). Viết DataGenerator.java để sinh ≥10,000 dòng dữ liệu vào các file CSV.	Class diagram (UML). CSV schema doc. Files dữ liệu mẫu ≥10k dòng.
T3	 Model Layer	Implement tất cả Entity classes (Seat, Fan, Ticket, Match...). Implement tất cả Enum. Implement abstract BaseEntity + toCsvLine/fromCsvLine. Implement CsvRepository<T> generic.	Model classes pass unit test parse/serialize CSV.
T4	 Repository Layer	Implement SeatRepository, FanRepository, TicketRepository, TransactionRepository. Test đọc/ghi file CSV chính xác. Implement tìm kiếm theo điều kiện (findByCondition với Predicate<T>).	Repository CRUD test pass. Đọc file ≥10k dòng < 500ms.
T5	 Controller Layer	Implement FanController (register, login, getMyTickets). Implement StadiumController (getSections, buildSeatMap). Implement BookingController — baseline NO_LOCK trước. Test booking đơn luồng.	Booking single-thread hoạt động đúng. Test case pass.
T6	 View Layer + MVC Wiring	Implement MainView, SeatMapView (ASCII seat map), BookingView, ReportView. Kết nối View ↔ Controller ↔ Model đúng MVC. Test luồng đặt vé end-to-end từ UI.	Demo booking flow từ đầu đến cuối trên console UI.
T7	 Synchronization Mechanisms	Implement FILE_LOCK (Java NIO FileLock). Implement SYNCHRONIZED (synchronized block trong Repository). Implement OPTIMISTIC (version field + conflict retry). Test từng cơ chế với 2-3 threads thủ công.	3 cơ chế hoạt động đúng. Không double booking với < 10 threads.
T8	 Simulator Tool	Implement SimulatorController với CountdownLatch + ExecutorService. Implement SimulatorView: in bảng so sánh ASCII. Chạy simulation 100–500 threads. Đo và ghi SimulationResult vào CSV.	Demo simulator 500 threads. Bảng kết quả đúng format.
T9	 Research & Report	Chạy full experiment: 1000 threads × 4 mechanisms. Vẽ biểu đồ so sánh throughput vs. double booking rate. Viết báo cáo Word: bối cảnh, thiết kế, kết quả, kết luận Research Question. Làm slide trình bày.	Báo cáo Word. Slide ≥ 15 trang. Biểu đồ so sánh.
T10	 AI Reflection & Nộp bài	Tổng hợp AI Log của từng thành viên. Viết AI Reflection: mô tả quá trình dùng AI, đánh giá chất lượng AI output, những gì AI làm tốt/kém, bài học rút ra. Review code, fix bug, polish UI.	AI Log file. AI Reflection trong báo cáo. ZIP hoàn chỉnh nộp.

 **Checkpoint quan trọng:** Tuần 4 (Repository) và Tuần 7 (Synchronization) là hai điểm kỹ thuật khó nhất. Nhóm cần dành buffer thời gian và không để dồn vào Tuần 9–10.

7. RUBRIC CHẤM ĐIỂM

Tổng điểm: 100%. Không có điểm bonus. Điểm nhóm = điểm chung. Điểm cá nhân = điểm nhóm × hệ số AI Reflection.

Tiêu chí	Trọng số	Mô tả chi tiết	Bằng chứng đánh giá
1. Decomposition	20%	Phân tích yêu cầu đúng thành các lớp có trách nhiệm rõ ràng (SRP). Phân chia Model / Repository / Controller / View không bị chồng lấn trách nhiệm. Xác định đúng tất cả Entity classes và quan hệ giữa chúng (1–N, N–N).	Review code: không có business logic trong View. Không truy cập CSV trực tiếp từ Controller.
2. Abstraction	15%	UML Class Diagram đầy đủ (attributes, methods, visibility, multiplicity) + 3 Flowchart (Booking, Sync, Simulator). Hierarchy rõ: BaseEntity → Entity. Generalization: CsvRepository<T> giải quyết CRUD cho mọi entity. Enum thay thế magic strings.	Nộp file diagram (.drawio / .png). CsvRepository<T> có thể dùng cho bất kỳ entity nào mà không cần sửa code.
3. Algorithm Design + Pattern Recognition	15%	CRUD đầy đủ cho Seat, Fan, Match (thêm/sửa/xóa/tìm kiếm). File I/O ổn định: load/save CSV đúng format, try-with-resources. Exception handling: ≥5 custom exceptions đúng ngữ cảnh. DataGenerator tự sinh ≥10,000 dòng CSV chính xác, không lỗi parse.	Demo tìm kiếm theo điều kiện. Run DataGenerator → đếm dòng từng file. Đọc file ≥10k dòng thành công.
4. Algorithm Design (Simulator)	20%	Simulator Tool chạy đúng: N threads đồng thời (CountDownLatch), thu thập kết quả đầy đủ. Implement đúng ≥3 cơ chế đồng bộ. Kết quả so sánh thực nghiệm hợp lệ: throughput (TPS) và double booking rate được đo chính xác. Biểu đồ / bảng so sánh rõ ràng.	Demo live: chạy 1000 threads với NO_LOCK → thấy double booking. Chạy SYNCHRONIZED → 0 double booking. Bảng số liệu đúng.
5. AI Reflection	30%	Mỗi thành viên nộp AI Log cá nhân (log raw các cuộc hội thoại với AI). Phân tích: AI đã giúp gì, không giúp gì, output AI có chính xác không, chỉnh sửa thể nào. Bài học: kỹ năng prompt tốt hơn, giới hạn của AI. Đánh giá mức độ phụ thuộc AI và hệ quả với việc học.	AI Log file có timestamp và nội dung thực. Reflection ≥500 từ/thành viên. Có ví dụ cụ thể (prompt → AI output → chỉnh sửa → kết quả).
TỔNG CỘNG	100%	AI Reflection 30% — đây là tiêu chí cá nhân quan trọng nhất, quyết định điểm của từng thành viên.	

7.1. Mức điểm chi tiết cho AI Reflection (30%)

Mức	Điểm	Mô tả
-----	------	-------

Xuất sắc	27-30%	AI Log đầy đủ, có phân tích sâu về chất lượng output AI, chỉ ra cụ thể lỗi AI mắc, thể hiện tư duy phản biện, bài học có thể áp dụng được.
Tốt	21-26%	AI Log đầy đủ, phân tích có chiều sâu, có ví dụ cụ thể, nhưng chưa có tư duy phản biện về giới hạn AI.
Đạt	15-20%	AI Log có nhưng thiếu chi tiết. Reflection chung chung, ít ví dụ cụ thể.
Không đạt	0-14%	Không có AI Log, hoặc Reflection chỉ là mô tả lại bài làm mà không đánh giá quá trình dùng AI.

7.2. Các lỗi bị trừ điểm

- **Logic nghiệp vụ trong View (vi phạm MVC):** -5%/lần phát hiện
- **Truy cập CSV trực tiếp từ Controller (không qua Repository):** -5%
- **DataGenerator không đủ 10,000 dòng tổng:** -5%
- **Simulator không dùng CountdownLatch (threads không đồng thời thực sự):** -8%
- **AI Log không tồn tại hoặc rõ ràng là giả mạo:** 0% phần AI Reflection
- **Không compile được:** 0% toàn bài

8. YÊU CẦU SẢN PHẨM NỘP

Cấu trúc ZIP nộp bài:

```
// Java
NHOM_XX_LAB211_TicketBooking.zip
├── src/                                ← toàn bộ source code Java (đúng cấu trúc MVC)
├── data/                              ← file CSV đã generate
│   ├── stadiums.csv
│   ├── sections.csv
│   ├── seats.csv                      (≥ 10,000 dòng)
│   ├── fans.csv
│   ├── matches.csv
│   ├── tickets.csv
│   └── transactions.csv              (kết quả simulation)
├── docs/
│   ├── report.docx                  ← báo cáo Word đầy đủ
│   ├── slide.pptx                  ← slide trình bày
│   ├── class_diagram.png           ← UML Class Diagram
│   └── flowcharts/                  ← 3 flowchart (Booking / Sync / Simulator)
├── ai_logs/
│   ├── member1_ai_log.md            ← AI Log cá nhân từng thành viên
│   ├── member2_ai_log.md
│   └── ...
└── README.md                       ← cách compile, chạy, và chạy Simulator
```

- Tên file ZIP: NHOM_XX_LAB211_TicketBooking.zip (XX = số nhóm)
- README phải có lệnh compile và chạy DataGenerator, sau đó chạy chương trình chính
- Nộp qua FPT University Learning trước deadline được thông báo
- Trễ 1 ngày: -10%. Trễ 2 ngày: -30%. Trễ ≥ 3 ngày: 0 điểm

Chúc các nhóm nghiên cứu và coding hiệu quả! 🍀

"Concurrency is hard. But understanding why it is hard — that is the real lesson."